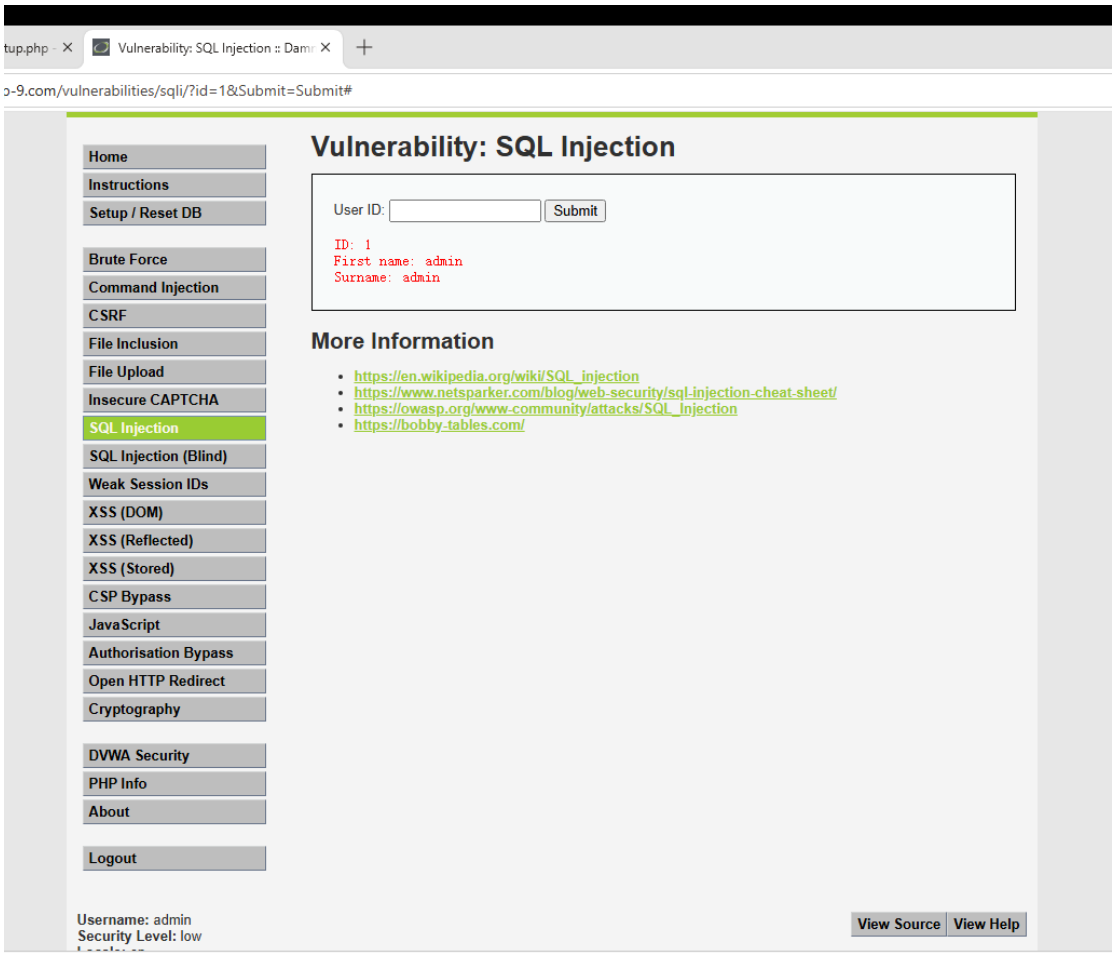
第三章 SQL 注入漏洞与防护——实验分析报告

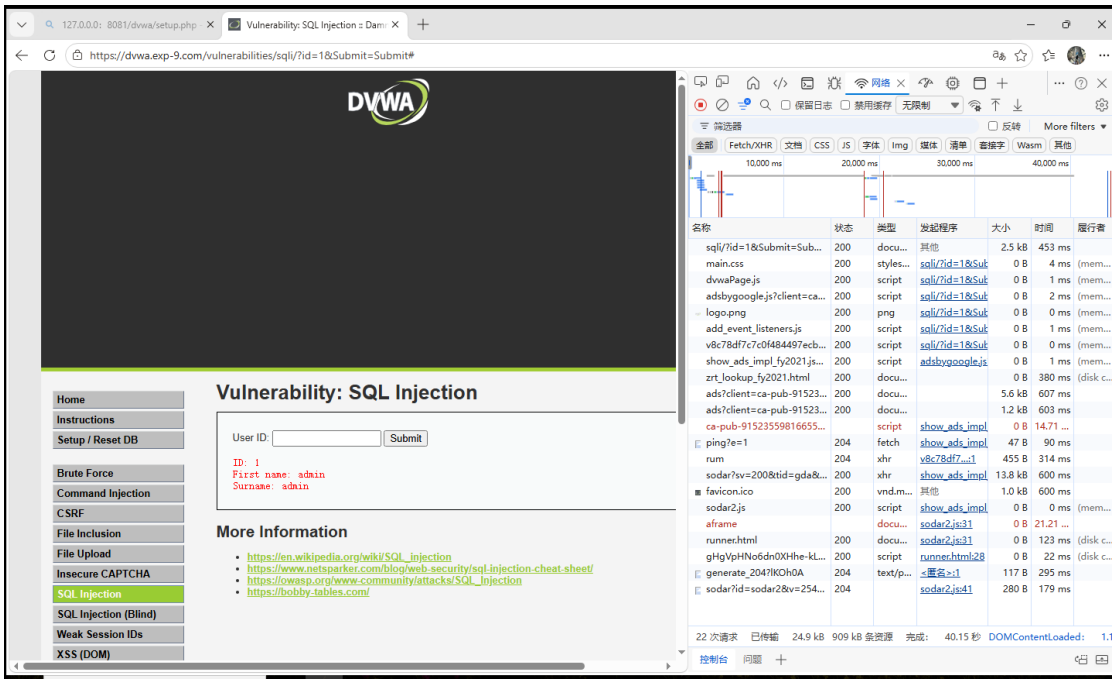
任务 1：数据库与查询条件基础说明

1. 页面中的输入为什么会与数据库查询有关



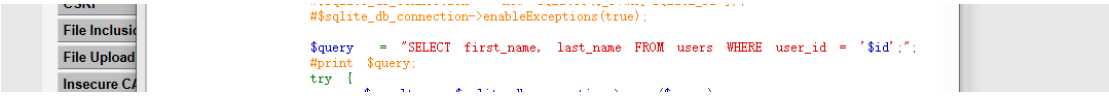
用户在 DVWA 的 SQL 注入页面输入 ID 为 1 并提交后，页面返回了数据库中 ID=1 的用户信息：First name: admin、Surname: admin。这说明页面的输入框会直接根据用户输入的 ID，去数据库中查询并返回对应数据，因此页面输入与数据库查询直接相关。

2. 一个最小业务链路是什么



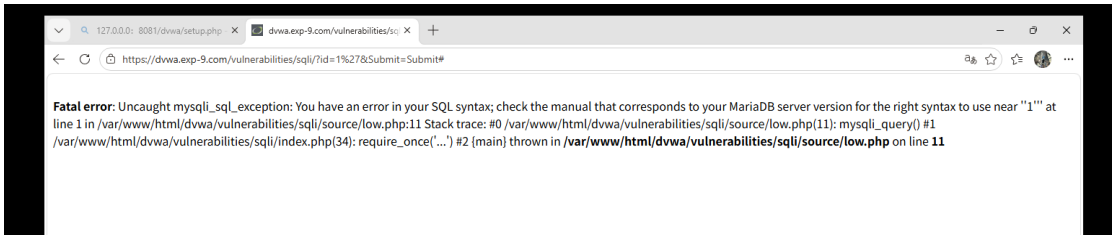
从浏览器抓包可以看到，用户输入的 id=1 会作为 GET 参数，随请求提交给后端的 sqli.php 接口，且请求状态码为 200，说明请求成功。完整的最小业务链路为：用户输入内容 → 前端提交请求参数 → 后端接收数据 → 拼接 SQL 语句 → 数据库执行查询 → 返回数据到页面，直观展示了用户输入如何传递到后端并关联数据库查询的过程。

3. 用户输入是如何进入查询条件的



从 DVWA 后端源码可以看到，程序直接将用户输入的$id 变量，通过字符串拼接的方式写入 SQL 语句的 WHERE 查询条件中，代码为：$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";用户输入未经过任何过滤，直接成为 SQL 查询条件的一部分，完全参与到了 SQL 语句的构造过程中。

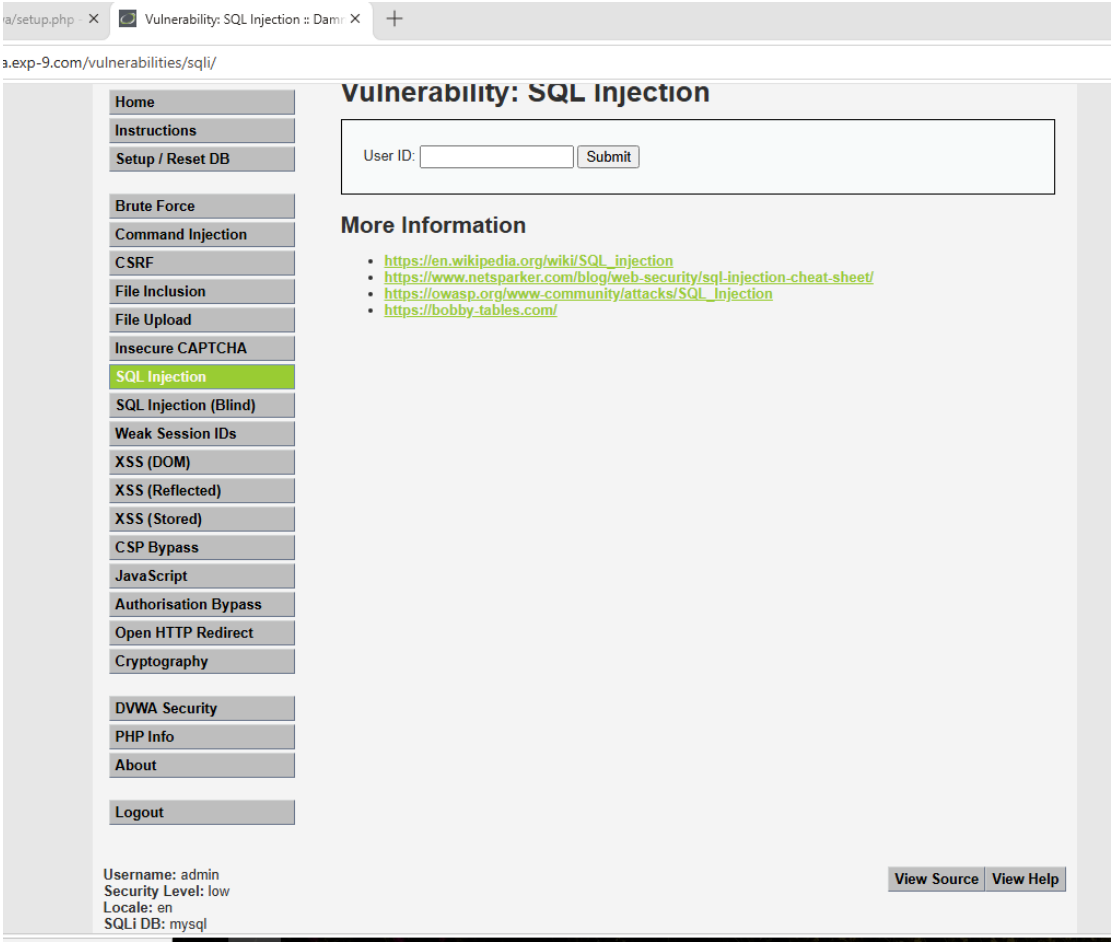
4. 为什么这为 SQL 注入创造了可能性



用户输入 1'后，页面直接返回了数据库的 SQL 语法错误信息：You have an error in your SQL syntax。这说明后端没有对用户输入的特殊字符进行过滤和转义，单引号直接破坏了原有的 SQL 语句结构，导致语法错误，证明了用户输入可以直接参与 SQL 语句构造，为 SQL 注入创造了条件。

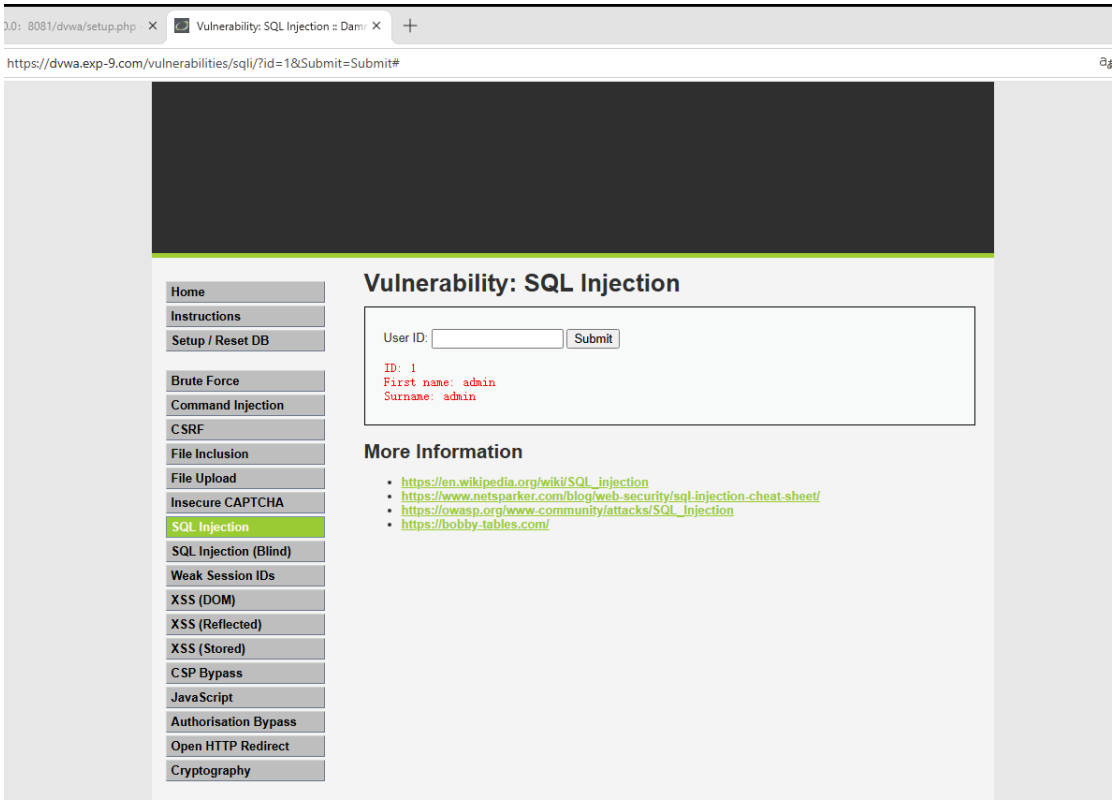
任务 2：DVWA SQL Injection Low 现象记录

1. 实验页面或实验目标



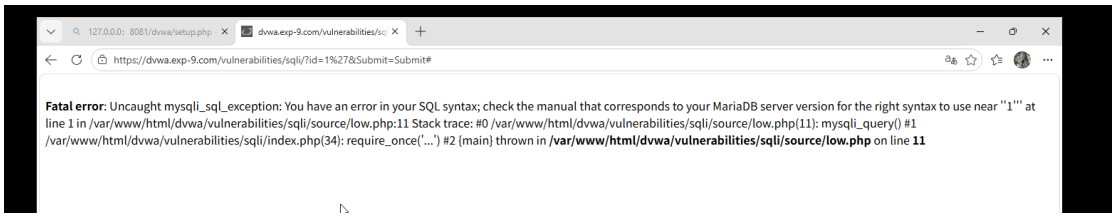
本次实验在 DVWA 靶场的 SQL Injection 模块进行，页面左下角明确显示安全级别为 low，环境符合实验要求。实验目标是观察和验证未过滤用户输入导致的 SQL 注入漏洞，对比普通输入与恶意输入的行为差异，理解“查询结构被污染”的核心问题。

2. 普通输入与异常输入的对比



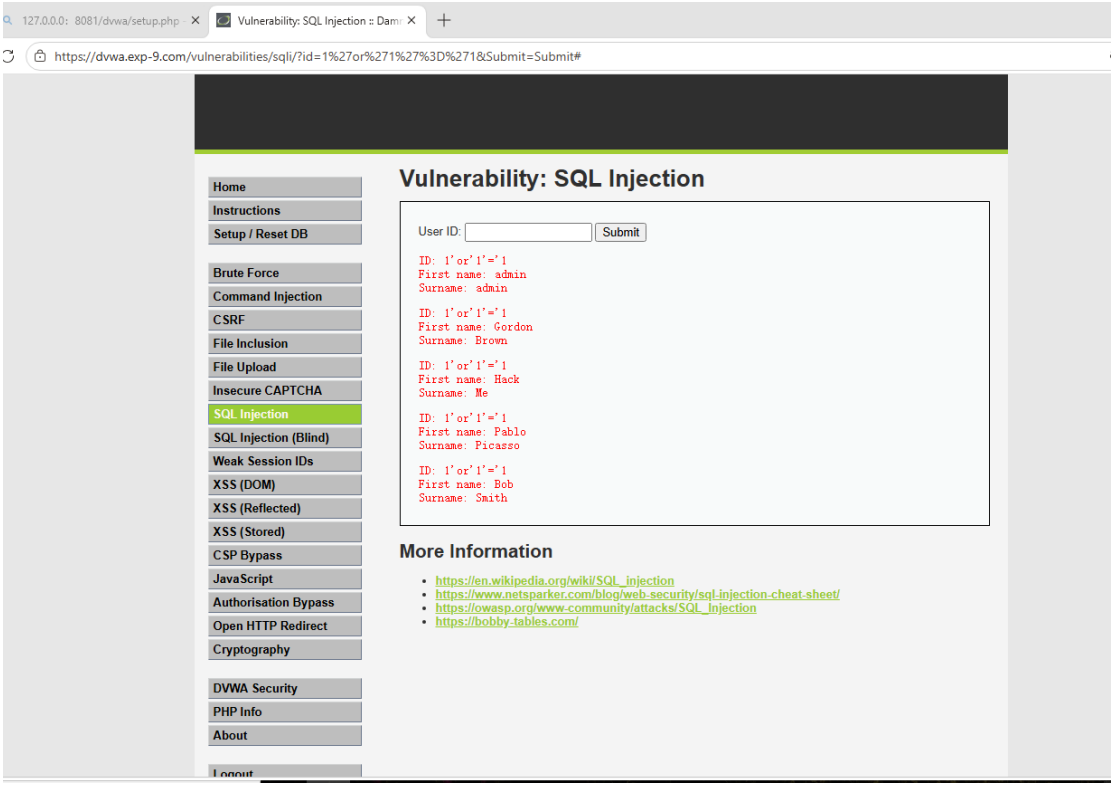
这是普通输入 1 时的查询结果。输入 ID 为 1 并提交后，页面仅返回了 ID=1 的用户信息：ID: 1, First name: admin, Surname: admin，结果与输入一一对应，符合“按 ID 查询单个用户”的正常业务逻辑，是预期内的正常查询行为。

3. 返回结果的差异



这是异常输入 1'时的查询结果。输入 1'后，页面直接返回了数据库的 SQL 语法错误信息：You have an error in your SQL syntax，说明单引号直接破坏了原 SQL 语句的单引号闭合结构，后端未对特殊字符进行过滤，用户输入直接参与了 SQL 语句的构造，这是 SQL 注入漏洞的直接证据。

4. 为什么这种差异不像“正常查询变化”



这是异常输入 1' OR '1'='1 时的查询结果。输入该恶意语句后，页面返回了数据库中 users 表的所有用户数据（admin、Gordon、Hack、Pablo、Bob），WHERE 条件被篡改恒为真，完全超出了“按 ID 查询单个用户”的业务预期，证明恶意输入成功篡改了 SQL 查询逻辑。

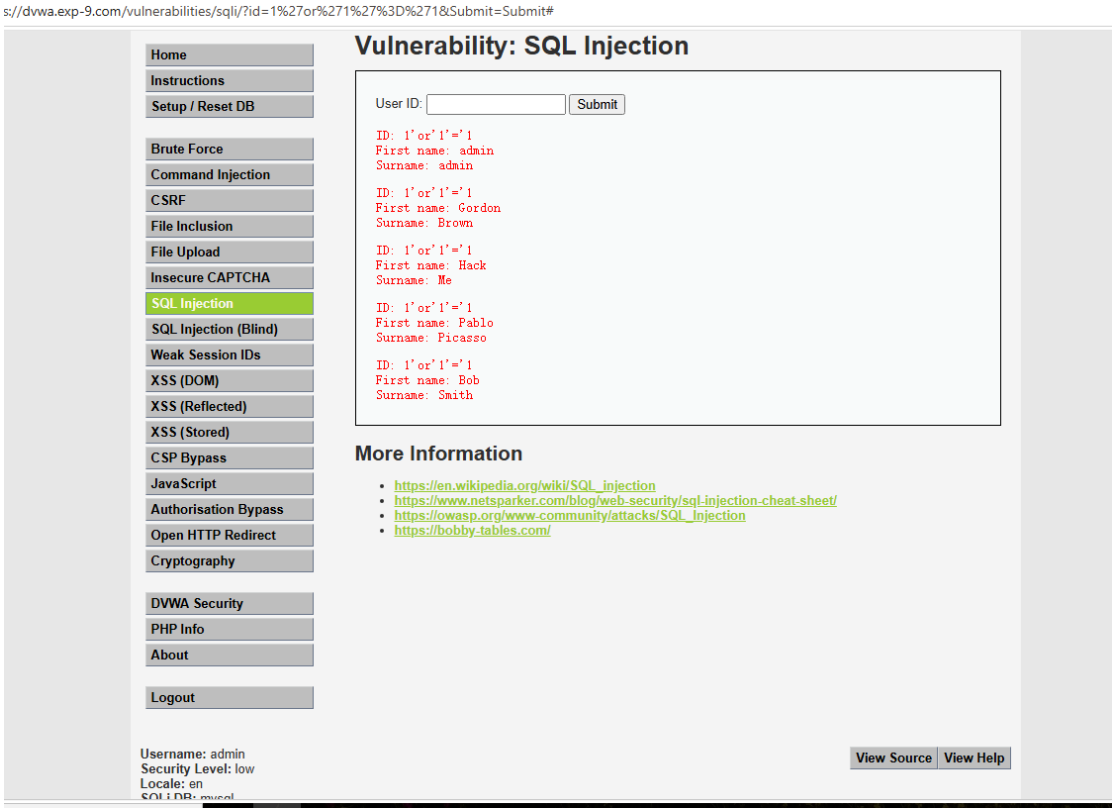
任务 3：机制分析与差异比较

1. 输入如何进入查询结构

```
$query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

在 DVWA Low 级别中，后端代码直接通过字符串拼接的方式，将用户输入的 $id 变量嵌入 SQL 语句中：$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";。用户输入未经过任何过滤或预处理，直接成为 SQL 语句的一部分，进入查询结构的 WHERE 条件中，最终被数据库解析并执行。

2. 为什么根因是“结构被污染”而不是“字符本身危险”



输入 1' OR '1'='1 后，页面返回了 users 表中的所有用户数据。这说明单引号和 OR 字符本身不是危险的，它们只是 SQL 语法符号；真正的问题是用户输入破坏了原有的 SQL 语句结构，让 WHERE 条件被篡改恒为真，执行了超出业务预期的全表查询，证明 SQL 注入的根源是“结构被污染”，而非字符本身危险。

3. Low 与 Medium 的差异在哪里

Low 级别防护逻辑：后端代码直接通过字符串拼接的方式，将用户输入的$id 变量嵌入 SQL 语句中：

```
$query  = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

Medium 级别防护逻辑：后端使用 mysqli_real_escape_string()函数对用户输入进行了转义，过滤了单引号等特殊字符：

```
$id = $_POST[ id ];

$id = mysqli_real_escape_string($GLOBALS["___mysqli_ston"], $id);
```

这种防护阻断了需要单引号闭合的字符型注入，但并未从根本上解决“用户输入直接拼接进 SQL 结构”的问题。由于输入框改为下拉菜单，前端限制了文本输入，但后端依然接收 URL 参数，理论上数字型注入（如 1 OR 1=1）依然可能绕过防护。

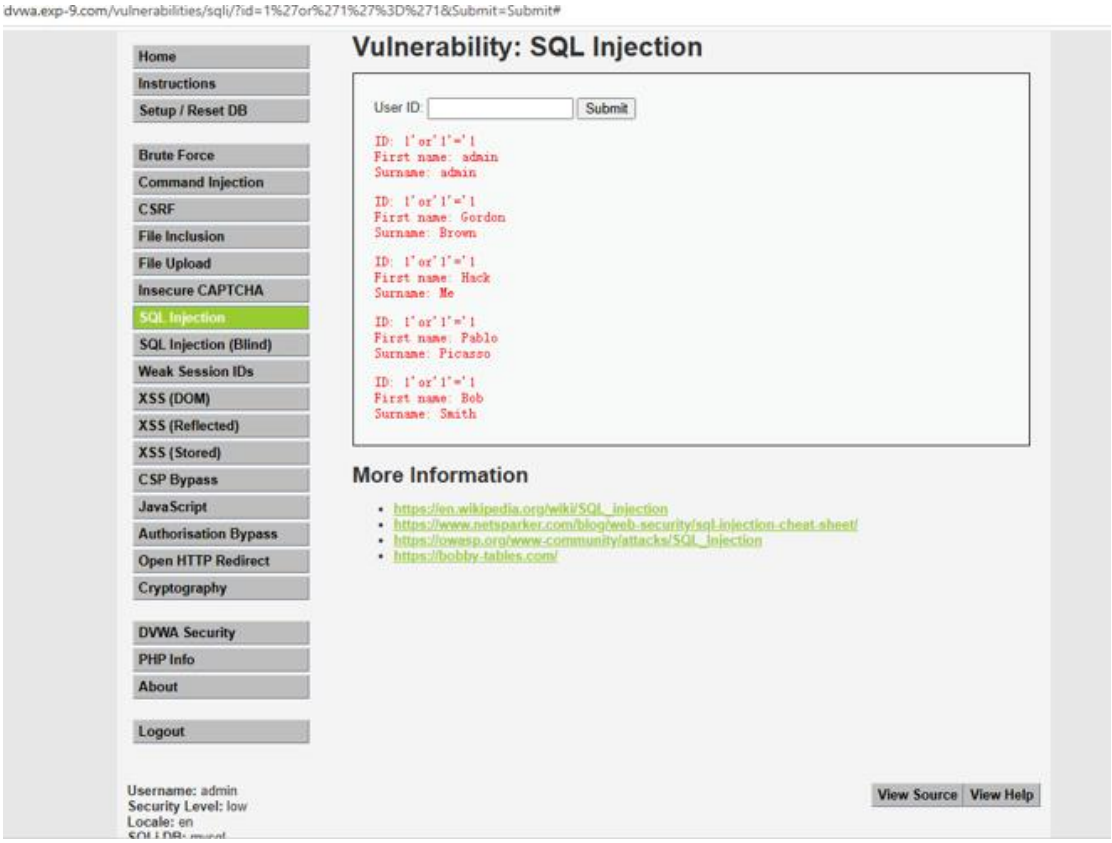
核心差异总结：

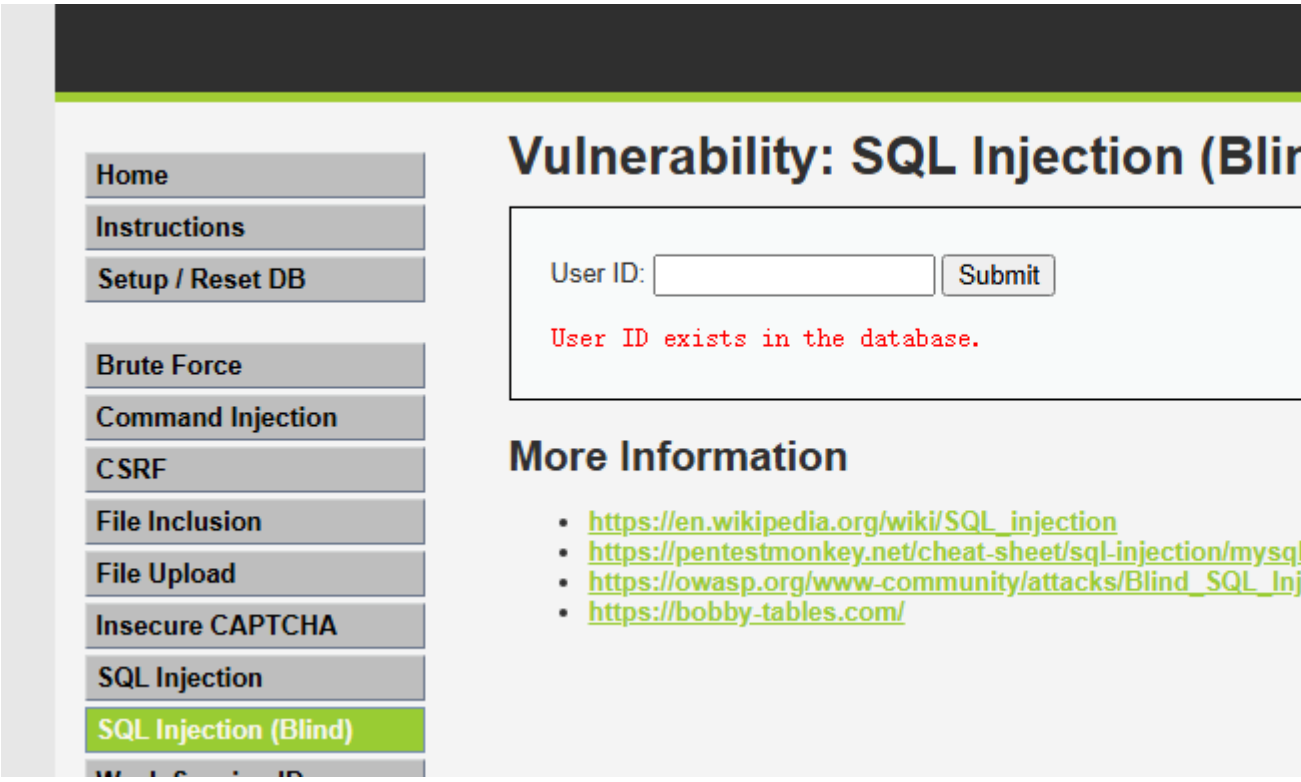
Low 级别：无任何防护，完全暴露在字符型注入风险中。

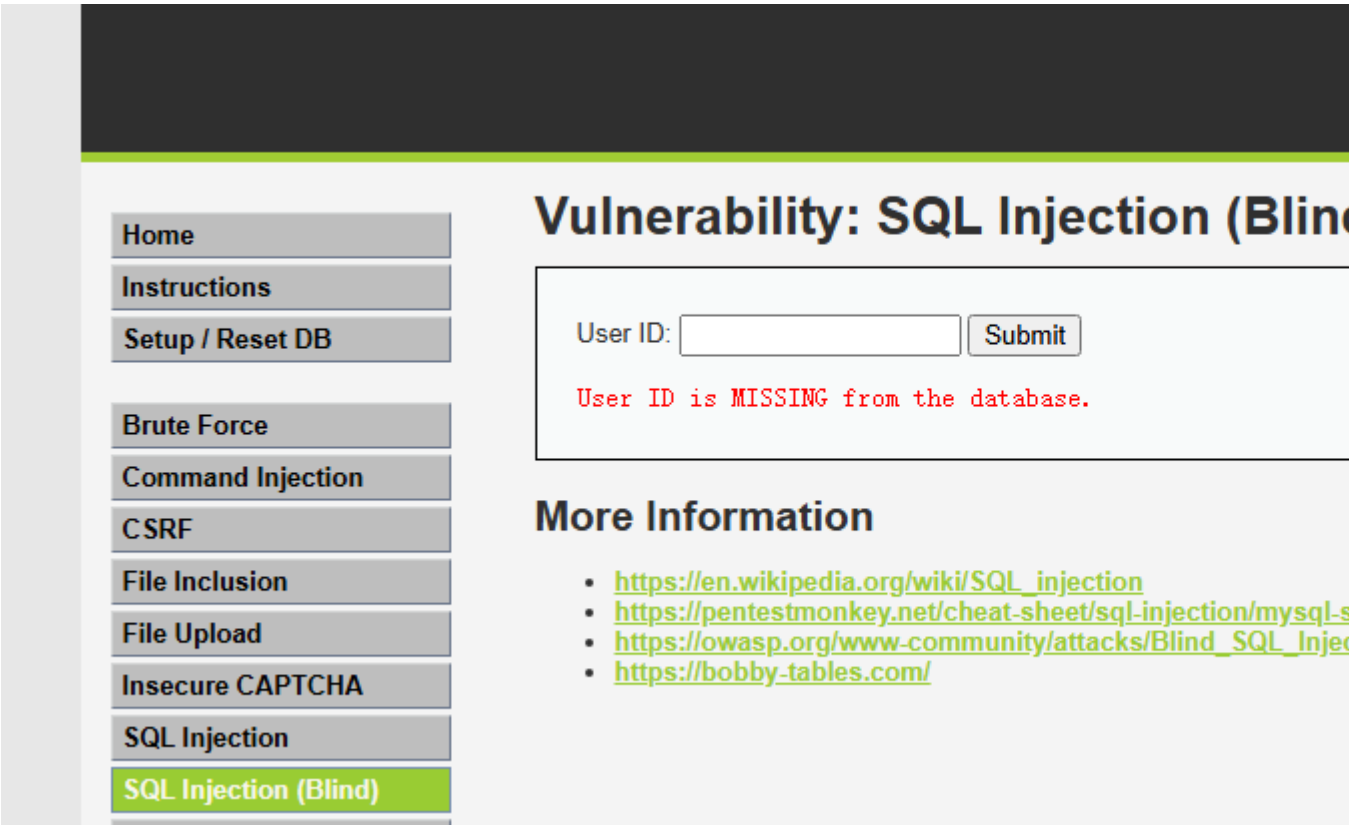
Medium 级别：仅通过转义单引号做了表层防护，阻断了字符型注入，但未修复

注入的根本原因 ——输入被直接拼接进 SQL 结构，依然存在数字型注入风险。

4. 普通 SQL 注入与 Blind SQL Injection 的差异在哪里





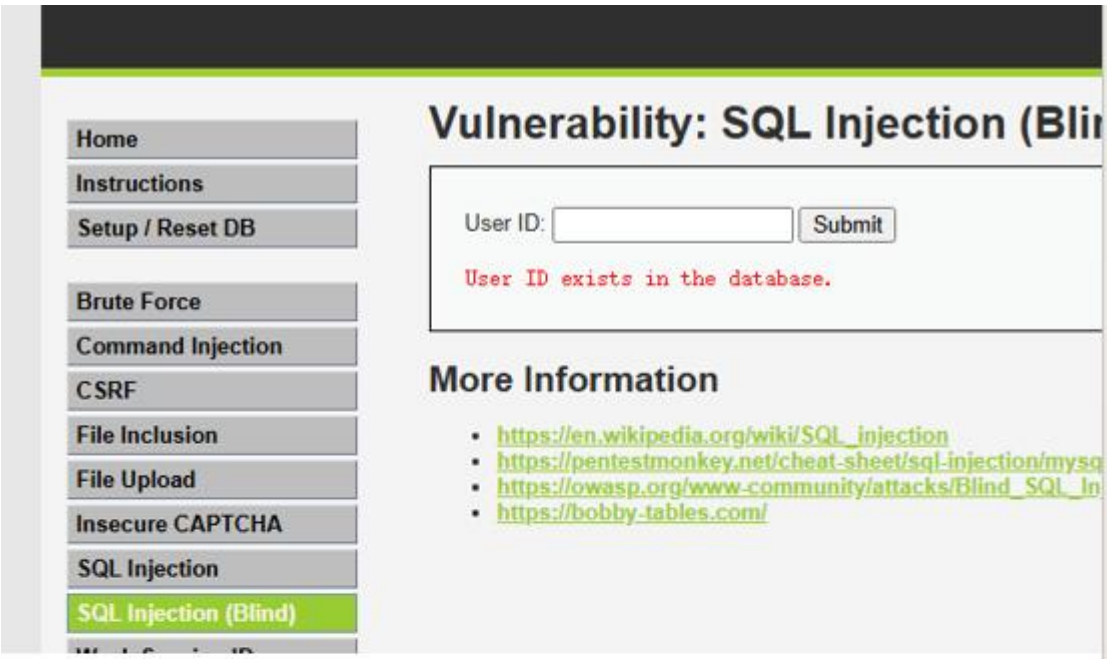


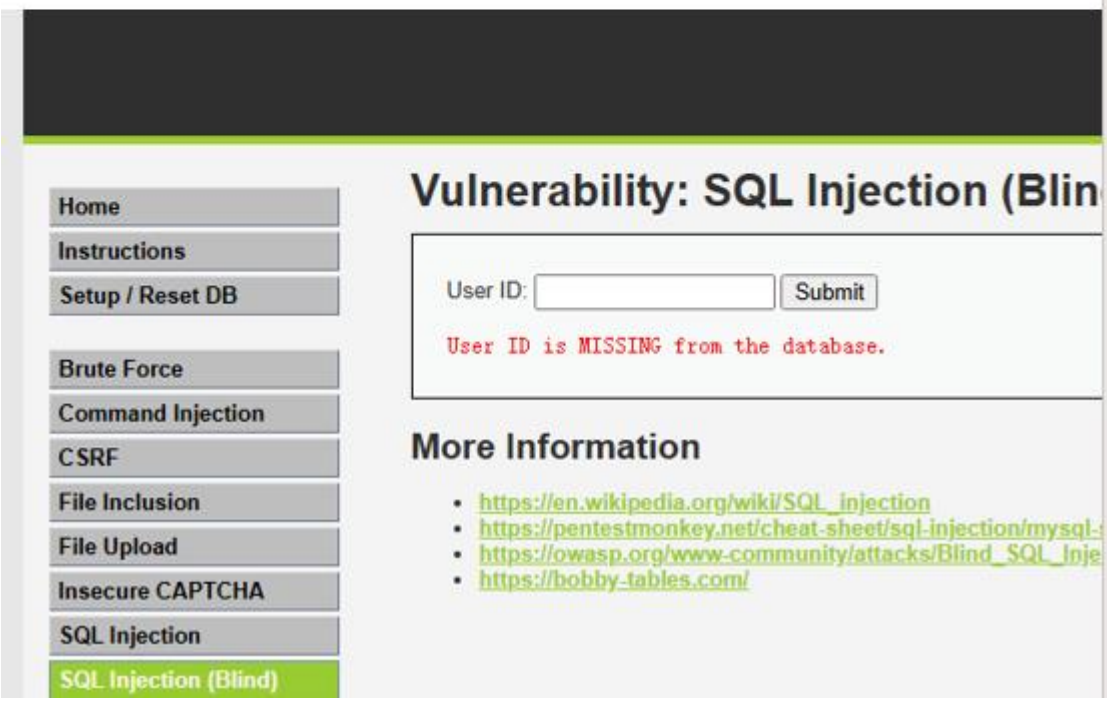
普通 SQL 注入：执行注入语句后，数据库的查询结果会直接回显在页面上，证据形式是“明确的数据内容”。

盲注（Blind SQL Injection）：页面不直接返回数据，只能通过“User exists”/“User

is MISSING”的行为差异来推断数据内容，证据形式是“间接行为差异”。

5. 为什么盲注和普通注入根因相同，但证据形式不同





上方：1' AND 1=1# → User exists（条件为真）

下方：1' AND 1=2# → User is MISSING（条件为假）

没有直接数据回显，只能通过页面的“存在 / 不存在”差异来判断注入结果，和普通 SQL 注入的直接数据回显形成了鲜明对比。

强制判断题

你必须明确表态并解释理由：

1. SQL 注入的根因更接近

A. 输入里有几个危险字符

B. 输入进入了不该进入的查询结构

选择 B. 输入进入了不该进入的查询结构

理由： SQL 注入的核心不是输入里的危险字符本身，而是用户输入被直接拼接进 SQL 语句，破坏了原有的查询结构，让输入从数据变成了可执行的指令。危险字符只是实现结构破坏的工具，根源是“结构被污染”，因此选 B。

2. Blind SQL Injection 和普通 SQL 注入的最大差异更接近

A. 根因不同

B. 证据形式不同

选择 B. 证据形式不同

理由： 两者的根源都是“输入拼接进 SQL 结构导致注入”，本质原因相同；最大差异是数据回显方式不同，普通注入直接回显数据 / 错误，盲注只能通过行为差异推断，证据形式完全不同，因此选 B。

任务 4：SQL 注入防护清单与解释

1. 参数化查询（预编译语句）

   它解决什么问题：直接解决 SQL 注入的根本成因—— 用户输入被当作 SQL 指令解析、破坏原有查询结构的问题。它将 SQL 语句的固定结构与用户输入数据完全分离，数据库只会把输入当作纯数据执行，无法篡改 SQL 逻辑。

   它控制哪一层：控制根因层，从源头杜绝注入的发生。

   它不能替代什么：不能替代输入校验、错误处理等辅助防护措施；也不能解决 SQL 逻辑错误、权限配置不当带来的其他安全风险。

2. 输入校验（白名单机制）

   它解决什么问题：通过限制输入格式、过滤特殊字符，拦截不符合业务规则的恶意输入（如非数字 ID、含'/OR 等 SQL 关键字的请求），减少恶意请求进入后端的机会。

   它控制哪一层：控制暴露面层，缩小可被利用的攻击面。

   它不能替代什么：不能替代参数化查询，因为复杂场景下校验规则容易被绕过，无法从根本上防止注入。

3. 错误处理（屏蔽数据库错误）

   它解决什么问题：关闭数据库错误回显，避免攻击者通过报错信息获取表名、字段名、数据库版本等敏感信息，防止攻击者利用这些信息构造更精准的注入语

句。

它控制哪一层：控制暴露面层，降低攻击者获取信息的能力。

它不能替代什么：不能防止注入本身，只是让攻击者更难利用漏洞；也不能替代日志审计和入侵检测。

4. 最小权限

它解决什么问题：限制 Web 应用连接数据库的账号权限，仅授予必要的 SELECT/INSERT 等权限，禁止使用 root 等高权限账号，即使发生注入攻击，也能限制攻击者能执行的操作，降低攻击造成的破坏范围。

它控制哪一层：控制后果范围层，减小漏洞被利用后的影响。

它不能替代什么：不能防止注入攻击的发生，只能限制攻击成功后的影响；也不能解决权限提升类攻击。

5. 可补充其他合理措施 部署 Web 应用防火墙（WAF）

它解决什么问题：通过识别并拦截含注入特征的请求（如' OR 1=1、UNION SELECT 等），在请求到达后端前进行拦截，作为额外的边界防护。

它控制哪一层：控制暴露面层，在网络层面增加一道防护屏障。

它不能替代什么：不能替代参数化查询，WAF 规则可能被绕过；也不能解决应用层的逻辑漏洞。

七、收口项

收口项 1：SQL 注入因果链/防护关系图

【输入点】

↓（用户可控参数直接传入）

【查询构造】→ 后端代码直接拼接用户输入到 SQL 语句

↓（未被安全处理）

【数据库解析/执行】→ 恶意输入被解析为 SQL 指令，篡改查询逻辑

↓（执行异常 SQL）

【结果差异或行为证据】

├── 普通注入：直接返回超出预期的数据（如全表信息）

└── 盲注：页面出现“存在/不存在”的行为差异

↓

【根因判断】→ 用户输入未被安全处理，直接参与 SQL 结构构造

↓

【防护控制点】

├── 控制点 1：参数化查询（控制根因层）→ 阻断“输入参与 SQL 结构构造”的路径

└── 控制点 2：输入校验（控制暴露面层）→ 拦截恶意输入，缩小攻击面

因果链：用户可控输入 → 直接拼接 SQL → 数据库解析恶意指令 → 结果异常 / 行为差异 → 根因：输入未被安全处理

防护对应：

针对“直接拼接 SQL”的根因 → 使用参数化查询，将 SQL 结构与用户输入分离

针对“用户可控输入”的暴露面 → 使用输入校验，拦截恶意请求

收口项 2：学生自评表

1. 我本章最清楚的一点是什么？

   我最清楚的是 SQL 注入的根本成因：用户输入被直接拼接进 SQL 语句，破坏了原有查询结构，被数据库解析为恶意指令执行。其中，参数化查询是从根源解决注入问题的核心防护手段，而转义过滤只能作为补充防护，无法完全替代。

2. 我最容易混淆的一点是什么？

   我最容易混淆的是“普通 SQL 注入”和“盲注”的差异。一开始误以为两者的攻击原理完全不同，后来才明白它们的根本成因都是 SQL 结构被污染，只是数据回显方式不同：普通注入直接返回数据，盲注只能通过“存在 / 不存在”的行为差异来判断结果。

3. 我本次作业中哪一段最需要教师复核？

   我本次作业中，关于“盲注如何通过行为差异判断注入结果”的部分最需要复核，尤其是在盲注的注入逻辑和防护有效性的理解上，希望老师能帮我确认分析是否正确。

4. 如果重做一次，我会优先改哪一部分？

   如果重做一次，我会优先完善防护方案的验证环节，补充参数化查询前后的效果对比截图，同时细化不同安全级别下的注入测试步骤，让实验过程和结论更直观、更严谨。

收口项 3：AI 输出审核与人工复核记录（必做）

1. 本作业哪些内容使用过 AI 辅助

   本次作业中，SQL 注入的因果链梳理、防护方案框架搭建、实验报告的文字组织与格式优化部分，使用了 AI 辅助生成基础内容和框架。

2. 哪些判断是你自己复核后保留的

   我复核并保留的核心判断包括：SQL 注入的根本成因、参数化查询是核心防护手段、普通注入与盲注的差异、不同安全级别的防护效果对比。这些结论都与我

的实验截图和操作结果一致，逻辑上也符合 SQL 注入的原理。

3. 哪些内容你发现 AI 输出不准确并进行了修正

   我发现 AI 最初提出的“输入过滤可以完全替代参数化查询”的说法不准确，修正为“输入过滤仅能作为补充防护，无法从根本上防止 SQL 结构被篡改，参数化查询才是核心防护手段”；同时补充了转义过滤无法防御数字型注入的实验验证细节。

4. 你最终如何确认结论和证据是一一对应的

   我对照实验操作截图和作业结论逐一验证：用 1' OR '1'='1 的全表数据截图验证了 SQL 结构被篡改的原理；用参数化查询前后的页面效果截图验证了防护方案的有效性；用盲注“存在 / 不存在”的行为差异截图验证了盲注的证据形式特征，确保每一项结论都有对应的实验证据支撑。